Edge cases (an unhandled routing value) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangGraph, plus making the failure message actually useful). Read Basic first even if you already know LangGraph — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

Take your Intermediate exercise's conditional edge and deliberately break the routing function so it sometimes returns a string that isn't one of the mapping's keys — like returning `"path_c"` when the mapping only knows `"path_a"` and `"path_b"`. Run it, and actually read the error LangGraph gives you, instead of guessing.

The question this exercise is really asking: does an unrecognized routing value fail loudly and clearly (good — you'll notice immediately), or does it silently do something unexpected, like defaulting to some node anyway (bad — you'd never notice until something was already wrong)?

Things to use:

- Your Intermediate exercise's graph, with `route` changed to sometimes return an unmapped string.
- A `try` / `except` around `graph.invoke(...)` so you can read the exception type and message without your script just crashing.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

`add_conditional_edges("from_node", route, {"path_a": "node_a", "path_b": "node_b"})` builds, internally, a lookup: call `route(state)`, then look up whatever it returned in that mapping to find the real next node. A plain dict lookup with a missing key raises `KeyError` — and that's exactly what happens here too: LangGraph doesn't quietly fall through to some default node, it lets the lookup fail, which surfaces as an exception at the moment that branch actually runs (**not** when you call `compile()` — the mapping isn't checked until the routing function's return value is actually looked up during a real `invoke()`).

The exact pieces:

- Change `route` so it returns `"path_c"` under some condition your test input can trigger — e.g. `if state["flag"] is None: return "path_c"` before the normal `if` / `else`.
- `try: graph.invoke({"flag": None, ...}) except Exception as e: print(type(e), e)` — catch broadly first, so you can see exactly what LangGraph actually raises, rather than assuming.
- Confirm it fails **at invoke time**, not at `compile()` time — compiling a graph doesn't run any routing function, so a bad mapping only shows up once you actually execute the branch that hits it.

Think about who actually reads the error LangGraph raises when this happens. It correctly fails loudly — that's the good news this exercise is meant to teach you to notice. But the message you get from the library's internal lookup failing doesn't necessarily say anything about *your* routing function, *your* state, or *why* the value was wrong — it just tells you a key lookup failed somewhere inside the library's branch-running code.

The real design question isn't just "does an unmapped value fail loudly" (yes) — it's "when it does, does the failure actually help whoever's debugging it, or just tell them something broke?"

The extra piece that helps:

- Add your own explicit check **inside** the routing function itself, before returning, and raise a specific, named exception — not a generic `ValueError` — so the failure is unmistakably "my routing function produced a bad value," not "some value error happened somewhere": `class UnhandledRouteError(Exception): ...`, then `raise UnhandledRouteError(f"route() produced an unhandled value: {result!r} for state: {state}")`. This fails at the same moment (still loud, still immediate), but the message — and the exception's own name — now point straight at your actual routing function, the actual bad value, and the actual state that caused it, instead of a generic lookup failure several stack frames inside LangGraph's own code.

```
# conditional_routing_practice.py — Edge cases section
class UnhandledRouteError(Exception):
    """Raised when route() produces a value the graph's mapping
    doesn't handle."""


def route(state: AgentState) -> str:
    if state["flag"] is None:
        result = "path_c"  # deliberately not in the mapping
    else:
        if state["flag"]:
            result = "path_a"
        else:
            result = "path_b"

    valid_paths = {"path_a", "path_b"}
    if result not in valid_paths:
        message = (
            f"route() returned {result!r}, "
            f"which isn't one of {valid_paths}"
        )
        raise UnhandledRouteError(message)
    return result
```

Difference between Basic and Intermediate: Basic establishes the fact this exercise is testing for — an unmapped value fails loudly, via a `KeyError`-style lookup failure, at invoke time rather than compile time. Intermediate doesn't change *whether* it fails — it changes *how useful the failure is*, by adding your own explicit check that raises a named `UnhandledRouteError` with a message naming your routing function, the actual bad value, and the state that produced it, instead of relying on a generic error from several layers inside the library.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
reuse the conditional-routing graph from the Intermediate exercise

change route so it can return "path_c"
    -- a string the mapping doesn't know about

wrap the invoke call in try/except

run it once with a normal input (flag=True) -- confirm it still works
run it once with the input that triggers "path_c" -- read the actual exception
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# conditional_routing_practice.py — Edge cases section
def route(state):
    if state["flag"] is None:
        return "path_c"
    if state["flag"]:
        return "path_a"
    return "path_b"

# ... same graph wiring as conditional_routing,
# mapping only has path_a/path_b ...

try:
    graph.invoke({"flag": None, "message": ""})
except Exception as e:
    print(type(e).__name__, "-", e)
```

Expected output if you run just this: nothing prints unless the exception actually gets raised — that only happens once `invoke` runs the broken branch. Run it and read exactly what type of exception you got and what its message says.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
same graph as the Intermediate exercise (conditional_routing), but:

def route(state: GraphState) -> str:
    if state["flag"] is None:
        return "path_c"   # not in the mapping on purpose
    if state["flag"]:
        return "path_a"
    return "path_b"

confirm:
    graph.invoke({"flag": True, ...})   -> works fine, unchanged
    graph.invoke({"flag": None, ...})   -> raises, at invoke time

print the exception's type and message -- is it a KeyError? does it
name "path_c" anywhere?
also confirm builder.compile() itself does NOT raise -- the bad
mapping isn't checked until a real invoke hits it
```

```
# conditional_routing_practice.py — Edge cases section
builder = StateGraph(GraphState)
# ... add_node / add_conditional_edges / add_edge, as in conditional_routing ...
graph = builder.compile()  # this line should NOT raise

try:
    graph.invoke({"flag": None, "message": ""})
except Exception as e:
    print(f"{type(e).__name__}: {e}")
```

Run this and read the real output before checking the Solution — the exact exception type and message matter here, that's the whole point of the exercise.

Once that's confirmed, make the failure message actually useful:

```
same setup, but route() itself checks its own result before returning,
raising a named exception instead of a generic one:

class UnhandledRouteError(Exception): ...
```

```
def route(state) -> str:
    if state["flag"] is None:
        result = "path_c"
    elif state["flag"]:
        result = "path_a"
    else:
        result = "path_b"
    valid_paths = {"path_a", "path_b"}
    if result not in valid_paths:
        message = (
            f"route() returned {result!r}, "
            f"which isn't one of {valid_paths}"
        )
        raise UnhandledRouteError(message)
    return result

run the same 2 invoke calls as Intermediate, and compare:
    the library's own KeyError-style message, vs.
    your own UnhandledRouteError's message

which one would actually help you debug this faster at 2am?
```

Try writing this yourself, then compare your own message quality against the Solution.

Difference between Basic and Intermediate: Basic confirms the fact — an unmapped routing value fails loudly, at invoke time, via whatever error LangGraph's internal lookup raises. Intermediate adds a check inside `route()` itself, raising a specific, named `UnhandledRouteError` so the failure carries a message written for a human debugging *your* graph, not a generic error from inside the library's branch-running code.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution